LAYA

Decision layer untuk aplikasi dan workflow Nagatech

Dokumen ini menjelaskan tujuan, manfaat, cara kerja, dan peluang penerapan Laya secara umum di lingkungan Nagatech.

Fokus utama: membantu sistem memilih jalur tindakan yang tepat dari input natural language, tanpa menggantikan database, API, aturan bisnis, atau approval manusia.

Status: bahan R&D; dan rekomendasi penerapan

1. Ringkasan eksekutif

Laya adalah lapisan keputusan ringan yang menerima konteks atau pertanyaan, lalu menghasilkan keputusan terstruktur. Keputusan ini dapat berupa kategori masalah, tujuan pengguna, tingkat urgensi, domain bisnis, atau workflow yang perlu dijalankan.

Dalam arsitektur AI, Laya berada di antara input pengguna dan sistem yang sudah ada:

Input user	Laya	Sistem Nagatech
Pertanyaan, keluhan, chat, event	Keputusan terstruktur dan bounded	API, database, workflow, atau petugas

Nilai utama Laya untuk Nagatech:

- Mengurangi routing manual pada chat, ticket, dan workflow operasional.
- Membuat keputusan awal lebih konsisten karena label dan output dibatasi.
- Memisahkan keputusan cepat dari proses berat seperti retrieval, query database, dan approval.
- Memungkinkan satu pola decision layer dipakai oleh banyak project tanpa menyalin seluruh agent AI.

Kesimpulan singkat: Laya bukan pengganti LangChain atau backend. Laya adalah komponen yang membantu sistem memilih apa yang harus dilakukan selanjutnya.

2. Tujuan dan cara kerja Laya

Tujuan Laya adalah membuat sistem dapat memilih jalur yang tepat dari input yang tidak terstruktur. Contohnya, satu kalimat pengguna dapat diarahkan menjadi pencarian knowledge, investigasi, pengecekan database, pembuatan ticket, atau eskalasi.

1	**Terima input**	Laya menerima pertanyaan, konteks sesi, project, dan metadata yang relevan.
2	**Baca definisi keputusan**	Sistem menentukan area, pilihan label, dan kriteria yang diperbolehkan.
3	**Hasilkan keputusan**	Laya mengembalikan label, confidence atau urgency, dan metadata keputusan.
4	**Jalankan workflow**	Aplikasi utama memanggil API, database, tool, atau petugas sesuai keputusan.
5	**Catat dan evaluasi**	Keputusan dicatat untuk audit, pengukuran akurasi, dan perbaikan definisi.

Contoh keputusan yang aman dan terbatas:

```
Input: "Laporan penjualan tidak sama dengan cash."

Keputusan Laya:
{
"area": "finance",
"operation": "sales_vs_cash_reconciliation",
"urgency": true,
"requires_database_check": true
}
```

Laya tidak boleh membuat query MongoDB bebas berdasarkan teks pengguna. Ia hanya memilih operation yang sudah diizinkan oleh aplikasi.

3. Mengapa Laya relevan untuk Nagatech

Workspace Nagatech memiliki beberapa jenis aplikasi: helpdesk, retail, finance, hospitality, perjalanan dinas, dan virtual assistant. Masing-masing mempunyai workflow serta sumber data berbeda, tetapi semuanya menghadapi masalah yang sama: input pengguna sering tidak rapi dan harus diarahkan ke proses yang benar.

Masalah umum	Tanpa decision layer	Dengan Laya
Pertanyaan ambigu	Agent menebak intent atau memanggil tool yang salah.	Intent dibatasi ke label yang sudah ditentukan.
Banyak workflow	Logika routing tersebar di banyak prompt dan controller.	Routing dipusatkan pada keputusan yang dapat diaudit.
Operasi berisiko	AI bisa menjawab sebelum data atau approval diverifikasi.	Laya menandai kebutuhan validasi, approval, atau eskalasi.
Perubahan project	Setiap project membangun pola klasifikasi sendiri.	Pola decision layer dapat digunakan lintas project dengan definisi berbeda.

Arsitektur yang direkomendasikan:

```
Frontend / WhatsApp / internal app
|
v
Laya decision service
| { area, operation, urgency, confidence }
v
API atau workflow project
|
v
Database, knowledge base, approval, atau petugas
```

Laya dapat dijalankan sebagai service terpisah. Project yang memakainya cukup mengirim request HTTP dan menerima JSON keputusan. Dengan cara ini, eksperimen Laya tidak harus mengubah seluruh arsitektur setiap project.

4. Kandidat project Nagatech

Daftar berikut adalah rekomendasi penerapan, bukan klaim bahwa semuanya sudah terintegrasi. Saat ini integrasi Laya yang sudah ada berada di NAVA; project lain masih menjadi kandidat implementasi.

Project	Peluang penerapan Laya	Prioritas
Buana Virtual Assistance	Memilih fungsi owner assistant: sales, stock, buyback, cash, debt, report, dan rekonsiliasi.	**Paling tinggi**
NAGAGOLD Online / NAGAGOLD AI	Routing masalah timbangan, stok, penjualan, receipt, buyback, dan laporan.	**Tinggi**
Finance Transaksional	Membedakan transaksi, summary, import, audit, approval, dan rekonsiliasi.	**Tinggi**
Finance Budget	Mengarahkan create budget, pemakaian, sisa budget, lampiran, dan approval.	**Menengah**
Finance Subscriber	Membedakan subscriber, invoice, payment, outstanding, dan rekap langganan.	**Menengah**
Hospitality OS	Memilih workflow front office, housekeeping, kitchen, maintenance, purchase, AP, dan lost and found.	**Menengah**
App Perjalanan Dinas	Mengarahkan pengajuan, approval, expense, reimbursement, dokumen, dan policy.	**Tahap dua**

Rekomendasi urutan R&D;

- Mulai dari Buana Virtual Assistance karena sudah memiliki pola percakapan owner dan banyak operasi bisnis yang jelas.

- Lanjutkan ke NAGAGOLD untuk menguji routing masalah operasional dan helpdesk.

- Uji Finance Transaksional setelah format rekonsiliasi, risk flag, dan validasi database siap.

- Setelah pola stabil, gunakan pendekatan yang sama pada budget, subscriber, hospitality, dan perjalanan dinas.

5. Contoh implementasi di beberapa project

Laya menghasilkan keputusan singkat. Sistem project tetap bertanggung jawab untuk mengambil data, memvalidasi hak akses, menjalankan proses, dan menyusun jawaban.

Input	Keputusan Laya	Aksi project
"Penjualan hari ini berapa?"	domain=sales; operation=sales_summary; period=today	Panggil service laporan penjualan.
"Stock barang saya sekarang berapa?"	domain=inventory; operation=stock_current	Panggil API stok realtime.
"Berat barang tidak muncul saat tambah barang."	area=inventory_weight; workflow=scale_connection_check	Tampilkan checklist timbangan atau buat ticket.
"Struk penjualan tidak keluar."	area=sales_receipt; workflow=printer_popup_check	Tampilkan panduan printer atau eskalasi.
"Kamar 305 AC rusak, tamu menunggu."	area=maintenance; urgent=true	Buat work order maintenance prioritas tinggi.
"Saya sudah pulang dinas, bagaimana reimbursement?"	area=business_trip; operation=submit_reimbursement	Buka workflow klaim dan cek dokumen.

Contoh kontrak output

```
{
"project": "finance-transaksional",
"area": "reconciliation",
"operation": "compare_detail_and_summary",
"confidence": 0.91,
"urgent": false,
"requires_database_check": true
}
```

Field seperti project, area, dan operation sebaiknya hanya menerima nilai dari allowlist. Jika confidence rendah atau label tidak dikenal, sistem harus memakai fallback aman seperti klarifikasi atau eskalasi.

6. Batasan, keamanan, dan pembagian tanggung jawab

Laya berguna ketika keputusan yang dibuatnya bounded dan dapat diverifikasi. Laya tidak boleh menjadi sumber kebenaran untuk data bisnis atau satu-satunya pengaman operasi berisiko.

Komponen	Tanggung jawab
Laya	Menentukan kategori, intent, area, urgency, dan kandidat workflow.
LangChain / agent	Mengatur reasoning, pemanggilan tool, grounding, dan penyusunan jawaban.
API project	Menerapkan aturan bisnis, authorization, validasi input, dan kontrak service.
Database	Menjadi sumber data resmi untuk angka, status, transaksi, dan histori.
Hindsight atau memory	Menyimpan konteks historis bila memang diperlukan dan sesuai scope customer/project.
Petugas / approver	Mengambil keputusan final untuk kasus sensitif, finansial, akses, atau pengecualian.

Aturan minimum untuk implementasi

- Jangan kirim credential, connection string, atau data sensitif yang tidak diperlukan ke Laya.
- Jangan biarkan Laya membuat free-form database query.
- Gunakan allowlist untuk area, operation, workflow, dan status.
- Jika hasil ambigu, minta klarifikasi atau arahkan ke petugas.
- Simpan audit log keputusan, tetapi batasi isi log sesuai kebijakan data.
- Pisahkan konfigurasi Laya tiap project agar label finance tidak tercampur dengan label hospitality.

7. Rencana POC yang disarankan

Agar keputusan penggunaan Laya tidak hanya berdasarkan asumsi, lakukan benchmark dengan pertanyaan nyata dan format evaluasi yang sama.

Tahap	Aktivitas	Output
1. Dataset	Kumpulkan 50-100 pertanyaan helpdesk atau operasional yang sudah dianonimkan.	Dataset dengan label expected.
2. Definisi	Tentukan label seperti greeting, knowledge, troubleshooting, investigation, escalation, dan domain bisnis.	Schema keputusan dan allowlist.
3. Baseline	Uji dataset ke NAVA atau routing yang berjalan saat ini.	Akurasi dan latency baseline.
4. Perbandingan	Uji dataset yang sama ke Laya, Lev, dan Jev jika tersedia.	Matriks hasil yang adil.
5. Review	Periksa salah routing, timeout, biaya, dan kemudahan deployment.	Daftar kelebihan dan risiko.
6. Keputusan	Pilih engine berdasarkan data, bukan asumsi atau popularitas.	Rekomendasi implementasi.

Metrik yang perlu dicatat

- Akurasi label dan ketepatan workflow.
- Rata-rata dan percentile latency.
- Jumlah fallback, klarifikasi, timeout, dan error.
- Biaya model atau kebutuhan resource.
- Kemudahan instalasi, deployment, observability, dan debugging.
- Dampak terhadap keamanan dan data project.

Rekomendasi akhir

Mulai dari satu pilot yang terukur, paling tepat Buana Virtual Assistance atau routing operasional NAGAGOLD. Setelah hasilnya stabil, pola decision layer dapat dibagikan ke finance, hospitality, dan project internal lain tanpa mengubah Laya menjadi sumber data bisnis.